

The Smallest Language Model

Episode 01 / Technical companion

Hussain Abuwala | LLM from Scratch | September 2026

This chapter develops a complete character-level bigram language model: define its outcomes, estimate a probability table from counts, generate text, evaluate predictions, and handle unseen transitions. The model is deliberately small enough that every numerical result can be checked by hand.

The training corpus is **anna, ava**. The worked evaluation example is **ana**; the constructed stress example **avna** exposes an unseen transition. These examples explain the mechanism. They are not a statistically representative benchmark.

A language model assigns probabilities. Generation chooses from those probabilities; evaluation scores the probabilities assigned to supplied targets.

Prerequisites

Counts, fractions, multiplication, and basic Python are sufficient. Logarithms are introduced before they are used. No calculus, neural networks, or PyTorch knowledge is assumed.

Reading map

Sections	Topic
1-2	Tokens, boundaries, counts and normalization
3-4	Generation and evaluation datasets
5-7	Sequence probability, logarithms and average NLL
8	Uniform and unigram baselines
9-10	Smoothing and choosing its strength
11-12	Limitations, implementation checks and exercises

This established n-gram model is the series' starting point. Later episodes add learned weights, shared representations, attention, and a small decoder-only Transformer. The repository also includes the editable Excalidraw canvas and a page-by-page canvas PDF.

1. Tokens, context and boundaries

A **token** is one unit processed by the model. Here, each ordinary character is a token. The ordinary vocabulary is {a, n, v}. We also introduce START and END as distinct special tokens; the printed word START represents one token, not five letters.

The **context** is the information used for the next prediction. A bigram model uses one previous token. A bigram is the pair consisting of that current token and the next token being predicted.

$$P(x_t \mid x_0, \dots, x_{t-1}) \approx P(x_t \mid x_{t-1})$$

This is a first-order Markov approximation: histories with the same most recent token share one probability row. It is an assumption of the model, not a claim that real language has only one-token dependencies.

We treat the two training names as independent examples:

```
START a n n a END
START a v a END
```

START selects the initial-character distribution. END lets the model assign probability to stopping at a particular position. We do not connect the end of one name to the beginning of the next.

Example	Ordered input-target transitions
anna	START→a, a→n, n→n, n→a, a→END
ava	START→a, a→v, v→a, a→END

There are **nine training predictions**. A name containing L ordinary characters contributes L+1 predictions: its characters and END. START supplies context and is not itself a target.

Allowed contexts and outcomes

Rows are [START, a, n, v]; columns are [a, n, v, END]. START is not predicted, and END is never used as the next context. In this convention START→END is allowed, so a smoothed model may generate an empty name. Excluding empty names is a different modeling choice and changes the allowed start-row outcomes.

A separate initial distribution or a separate length model could serve boundary-related roles in other designs. START and END are the explicit convention used here.

2. Training by counting and normalizing

Let $C(i,j)$ be the number of times target j follows context i in the training data. Let $C(i)$ be the sum of all outgoing counts in row i .

Current / next	a	n	v	END
START	2	0	0	0
a	0	1	1	2
n	1	1	0	0
v	1	0	0	0

Direction matters: $C(a,n)$ and $C(n,a)$ refer to different observations. For the unsmoothed model, divide each count by its row total:

$$\hat{P}(j | i) = \frac{C(i,j)}{C(i)}, \quad C(i) = \sum_j C(i,j)$$

For context a, the counts [0,1,1,2] sum to four. Normalization gives [0,0.25,0.25,0.50]. This is equivalent to a bag of four tickets: n, v, END, END.

Current / next	a	n	v	END
START	1.00	0.00	0.00	0.00
a	0.00	0.25	0.25	0.50
n	0.50	0.50	0.00	0.00
v	1.00	0.00	0.00	0.00

Every row contains nonnegative probabilities summing to one. The table, token labels, and boundary rules constitute the finished model. Counts may also be saved for inspection or to change smoothing.

These row frequencies are the **maximum-likelihood estimate** for the unrestricted unsmoothed bigram table. They minimize training NLL within that model class. All rows in this toy corpus have observations; an unobserved row would require an explicit policy rather than division by zero.

Implementation convention

The compact table is 4×4, but the combined token inventory has five symbols. A single global ID map cannot directly index both compact axes. Use dictionaries keyed by symbols or separate row and column maps in the orders shown above. IDs are labels, not measurements of character similarity.

3. Generation: greedy decoding and sampling

Generation starts with START. Look up the current row, select a next token, append an ordinary character to the output, and use it as the new context. When END is selected, stop. The probability table remains unchanged throughout.

Greedy decoding

Greedy selects the largest probability in the current row. From START it chooses a; from a it chooses END, whose probability 0.50 exceeds both 0.25 alternatives. The result is the one-letter name a.

```
START → a → END
```

Greedy is deterministic given a fixed model, initial context, and tie-breaking rule. It makes the locally highest-probability choice; it does not generally guarantee the most probable complete sequence or the most useful output.

Weighted sampling

Sampling selects outcomes according to their probabilities. It is not uniform selection over token types. At a, imagine drawing from the tickets n, v, END, END. Replace the ticket before a future draw from that row.

One possible sampled path is:

```
START → a → n → a → END
          1   0.25 0.50 0.50
output: ana
```

The 0.25 branch can be selected even though END is the most likely outcome at a. Sampling provides variety; it can also produce awkward strings. A fixed random seed helps reproduce an implementation's sampling run.

Choosing rule	Useful property	Limitation
Greedy	Simple, repeatable local choices	Little variety; locally optimal only
Sampling	Explores different possible outputs	Can produce poor or repetitive results

Both methods need a practical maximum-length guard. If the limit is reached before END, mark the sample as truncated rather than pretending the model chose to stop. These safeguards concern generation; they do not change how held-out NLL is calculated.

Changing the decoding rule changes the output path. It does not retrain the model or improve the underlying probability table.

4. Training, validation, testing and evaluation

Evaluation is the activity of measuring predictions. **Validation** and **test** describe two purposes for held-out evaluation data.

Split	Role
Training	Supplies transition counts used to fit the table
Validation	Compares development choices, including smoothing k
Test	Measures a chosen version after development choices are fixed

Split independent examples before counting. Fix preprocessing and splitting rules rather than repeatedly changing them until the test result looks favorable. Duplicate handling depends on the intended task; document whether repeated identical names can appear across splits.

For this project's training-only character vocabulary, report any held-out characters that cannot be represented. Smoothing cannot solve a missing-token problem. A predeclared character inventory, an explicit unknown token, or byte-level input are alternative designs.

Why score ana if we already generated it?

Generating a string does not add it to the training data. The model was fitted only to anna and ava. We reuse ana to explain the same four transitions from two perspectives:

Generation	Evaluation
Chooses the next token	Receives the next token from existing text
Builds a new path	Scores a supplied path
Does not have a target answer key	Uses actual preceding tokens and actual targets

At context a in the evaluation example, score the supplied target n even though greedy would choose END. Do not follow generated choices during this calculation.

The single ana example is hand-selected for teaching. Selecting only attractive generated names as an evaluation set would bias the result. Real comparisons use an independently selected collection of held-out examples.

When is the model fixed?

Generation can happen during development. After validation selects settings, freeze that particular table for testing. Development may resume afterward, but repeatedly tuning to the same test set turns it into additional validation data. A fresh untouched test set is a cleaner final assessment if test feedback influenced the redesign.

5. Sequence probability: why multiply?

A complete path requires every conditional choice along it. The probability chain rule expresses this as a product. Let $x_0 = \text{START}$ and $x_{L+1} = \text{END}$. Under the bigram approximation:

$$P(x_1, \dots, x_L, \text{END} \mid \text{START}) = \prod_{t=1}^{L+1} P(x_t \mid x_{t-1})$$

For ana, the four probabilities come from the fixed table:

Required transition	Probability
START→a	1.00
a→n	0.25
n→a	0.50
a→END	0.50

$$P(\text{ana}, \text{END}) = 1 \times \frac{1}{4} \times \frac{1}{2} \times \frac{1}{2} = \frac{1}{16} = 0.0625$$

Each step keeps a fraction of the paths that still match. A quarter of attempts choose n after a; half of those choose a; half of the remaining matching paths stop. These are conditional probabilities, not an independence assumption about successive characters.

Expected frequency is not a quota

Imagine N independent generation attempts, each restarted at START and sampled from the same table. If K counts complete outputs equal to ana, its expected value is $N/16$. For $N=16$, the expected count is one, but the observed count may be zero, one, two, or more.

For $N=16,000$, roughly 1,000 matching outputs is a useful intuition. As repetitions increase, the observed proportion tends toward $1/16$. The proportion becomes more stable; the absolute difference from the expected count need not shrink.

The model does not compensate for previous attempts. Even after many failures to produce ana, the next independent attempt still has probability $1/16$. Similarly, a fair coin need not alternate heads and tails to maintain a 50% probability.

The canvas's 16→16→4→2→1 sequence shows expected matching counts, not guaranteed intermediate results.

6. Logs: keeping tiny probabilities usable

Multiplying many factors below one makes a probability extremely small. A thousand factors of 0.1 give a mathematically nonzero value:

$$0.1^{1000} = 10^{-1000}$$

That is outside the representable positive range of ordinary floating-point types. A computation may store zero instead, losing the distinction between different tiny probabilities. This is **numerical underflow**.

A logarithm asks for an exponent

Ten cubed is one thousand, so log base ten of one thousand is three. Natural logarithm, $\ln$, uses the base e , approximately 2.718. Its crucial property here is:

$$\ln(ab) = \ln(a) + \ln(b)$$

Instead of forming a potentially underflowing product, take the log of each individual probability and sum those logs. Taking the log after a product has already rounded to zero does not restore lost information.

For our example:

$$\ln P(\text{ana}, \text{END}) = \ln(1) + \ln(0.25) + \ln(0.5) + \ln(0.5) \approx -2.772589$$

Negative log-likelihood

For positive p at most one, $\ln(p)$ is non-positive. Its negative is a nonnegative penalty that grows as the probability assigned to the actual target decreases:

$$\ell(p) = -\ln p$$

Probability on actual target	Penalty in nats
1.00	0.000000
0.50	0.693147
0.25	1.386294
0.10	2.302585
0.01	4.605170

This penalty is often described as surprise. It is a mathematical score, not a feeling or an accuracy percentage. Higher probability on the supplied target produces lower surprise.

7. Average NLL and its units

Total NLL accumulates a penalty for every prediction. A longer example gives the model more opportunities to accumulate penalties. To measure typical per-prediction performance, divide by the number of evaluated transitions.

$$\overline{\mathcal{L}} = -\frac{1}{N} \sum_{t=1}^N \ln P(\text{target}_t \mid \text{context}_t)$$

N includes predictions of END and excludes START as a target. Across a line-based dataset, construct transitions within each example, sum all penalties, and divide by all transitions.

Transition in ana	Probability	Negative log penalty
START→a	1.00	0.000000
a→n	0.25	1.386294
n→a	0.50	0.693147
a→END	0.50	0.693147

$$\overline{\mathcal{L}}(\text{ana}) = \frac{2.772589}{4} \approx 0.693147 \text{ nats/prediction}$$

Four predictions with total penalty two and eight predictions with total penalty four both have average penalty 0.5. Averaging removes the simple accumulation due to prediction count; it does not make unrelated datasets equally difficult.

For ana and av, there are four and three transitions. Aggregate the seven penalties and divide by seven. Averaging the two per-name averages equally would instead give each name equal weight, answering a different question.

Nats, bits and related terminology

Natural logs produce **nats**; base-two logs produce **bits**. Divide nats by $\ln(2)$ to convert to bits. A 50% target probability has penalty 0.693147 nats, or one bit. Average NLL of 0.693 is not 69.3% accuracy.

Empirical average NLL is the data-to-model conditional cross-entropy for these targets. Perplexity is $\exp(\text{average NLL})$, a different scale for the same score. Neither additional term is needed to implement this episode.

Compare models on the same held-out data, vocabulary, tokenization, boundary rules and denominator. Changing from characters to subword tokens changes what one prediction means. Here END is counted, so “bits per prediction” is more precise than “bits per ordinary character.”

8. Baselines: interpreting the score

The value 0.693 becomes more useful when compared with simpler models on exactly the same ana example. The following baselines use the same four allowed target types and include END in scoring.

Uniform baseline

Assign probability one quarter to every allowed next token, independently of context. Every target receives the same penalty:

$$\overline{\mathcal{L}}_{\text{uniform}} = -\ln(1/4) = \ln 4 \approx 1.386294$$

Unigram baseline

Count each training target without regard to its preceding context. The nine targets in *anna* END and *ava* END give counts a:4, n:2, v:1, END:2. START is not a target and is excluded.

The unigram distribution is [4/9, 2/9, 1/9, 2/9] at every prediction step. For *ana*'s targets a, n, a, END:

$$\overline{\mathcal{L}}_{\text{unigram}} = -\frac{2\ln(4/9) + 2\ln(2/9)}{4} \approx 1.157504$$

Bigram model

Our table uses the preceding token to select the distribution. It assigns probabilities 1, 0.25, 0.50, 0.50 along the supplied *ana* path, giving average NLL 0.693147.

Model	Information used	Average NLL
Uniform	Equal chances	1.386294
Unigram	Overall target frequency	1.157504
Bigram	One previous token	0.693147

On this example, knowing token frequency helps, and conditioning on the previous token helps further. This is not a guarantee that a bigram wins on every dataset. The larger build experiment must recompute all three scores on a held-out collection.

Unigram asks “How common is n overall?” Bigram asks “How common is n after this current token?”

The unigram assigns n probability 2/9 everywhere. The bigram assigns it 1/4 after a, 1/2 after n, and zero after v before smoothing.

9. Smoothing: unseen does not imply impossible

The constructed example *avna* uses known tokens but contains the unseen pair *v*→*n*. Its original probability is zero. A zero factor makes the complete sequence probability zero, even when all other transitions have positive probability.

The real-valued logarithm of zero is undefined. We use the extended loss value infinity because:

$$\lim_{p \rightarrow 0^+} -\ln p = +\infty$$

Infinity is not a particular huge number. The penalty has no finite upper bound as probability approaches zero. It expresses that the model assigned no chance to an event that occurred. One such term makes aggregate NLL infinite. This is distinct from floating-point underflow: this table actually contains zero.

Add-one smoothing

Add one pseudo-count to every allowed outcome before normalization. For the v row:

Stage	a	n	v	END
Observed	1	0	0	0
Add one	1	1	1	1
Adjusted count	2	1	1	1
Probability	0.40	0.20	0.20	0.20

The new total is five. These extra counts are not claims of new observations; they deliberately soften the estimate from limited evidence. Giving n some probability takes probability away from a. Probabilities must still sum to one.

Add-k smoothing

With A allowed target types in a row, the general rule is:

$$P_k(j | i) = \frac{C(i,j) + k}{C(i) + kA}, \quad k \geq 0$$

Here A=4 in every row. For k=0.1, the v counts become [1.1,0.1,0.1,0.1], with total 1.4. At k=0 we recover the observed-row estimate; as k becomes very large, rows approach uniform distributions.

Apply the rule across all rows, including START. It gives known-but-unobserved pairs positive probability when k>0; it cannot invent an out-of-vocabulary token. Add-k is a simple teaching method, not a claim about the best smoothing method for every application.

10. Choosing k and understanding the tradeoff

k is a **hyperparameter**: a development setting chosen around the count-fitting process. Try a small candidate set such as 0, 0.001, 0.01, 0.1 and 1, evaluate each on the same validation set, and select the lowest validation NLL. The list is illustrative, not a universal prescription.

Do not select k using test results. After choosing it, freeze that version for the final test measurement. Generation can be used to inspect behavior throughout development, but appealing samples are not a substitute for held-out scoring.

Smoothing does not always improve validation loss

The ana example already has only observed transitions. Smoothing makes its score worse for the positive strengths below. The avna stress example, in contrast, goes from infinite to finite loss.

k	Training NLL	NLL on ana	NLL on avna
0	0.616131	0.693147	infinite
0.01	0.625968	0.700555	1.489322
0.1	0.702274	0.759913	1.135742
1	1.026006	1.039721	1.153664
10	1.319101	1.319529	1.337820

All entries above smooth the entire table. Training loss averages nine transitions; each evaluation score uses its own transition count. These two constructed evaluation words are explanatory probes, not a representative validation set for choosing production settings.

For add one, ana's probabilities change at every relevant context:

$$P_1(\text{ana}, \text{END}) = \frac{3}{6} \times \frac{2}{8} \times \frac{2}{6} \times \frac{3}{8} = \frac{1}{64}$$

Its average NLL is therefore $\ln(64)/4$, approximately 1.039721. Replacing only the zero factor in avna while leaving all other factors unsmoothed would describe a different model.

Because normalized counts minimize the unrestricted table's training NLL, smoothing cannot improve that optimum. It strictly worsens training NLL for this corpus; an already uniform observed row can remain unchanged. A U-shaped validation curve or a positive best k is not guaranteed. Measure the behavior rather than drawing the desired curve in advance.

11. The limitation: too little context

These histories all end in the same most recent character:

```
a  
ana  
ava
```

Our bigram model queries the a row in every case. It cannot condition on how it reached a, how long the name is, or how many times a pattern has repeated. The program may retain the full output for display, but this model uses only the most recent token for prediction.

More data can improve the estimates in a row. It does not increase this architecture's context length. Smoothing changes the probabilities, not the information available to the model.

Local support is not global coherence

The output annava uses only transitions observed in the training corpus, including its boundaries. It can still be an awkward complete name. After each n, the model returns to the same n row; it cannot distinguish the first repetition from the fifth.

The two limitation slides therefore show one cause and one consequence: one-token context makes distinct histories indistinguishable, which permits locally supported but globally awkward outputs.

What more context costs

A trigram uses two previous tokens and can distinguish na from va. With V ordinary symbols, the number of possible length-m contexts grows as V to the power m, before boundary details. For V=30, one-, two-, and three-character contexts give 30, 900, and 27,000 possibilities. Sparse storage avoids allocating every unused row, but does not remove the lack of observations for rare contexts.

Our independently estimated rows also cannot share evidence about similar contexts. A full one-hot-to-logits neural bigram still has separate rows; changing counts into learned weights alone does not fix this. Shared representations are introduced later with embeddings and the MLP.

The immediate sequence is: implement this bigram, try two-character context in Chapter 2, then introduce learned bigram weights in Chapter 3. Additional context and neural representations can help; neither automatically guarantees coherent output.

12. Build checks, exercises and references

Checks for the coding video

- Reproduce the two-name corpus first: nine counts, the probability table, greedy output a, and $NLL(ana)=0.693147$.
- Verify that each row sums to one; do not use five global token IDs directly as both axes of the compact 4×4 table.

- Count each example separately and score END. Keep training counts unchanged during generation and evaluation.
- Use an explicit maximum generation length, deterministic split rules and documented handling of unknown characters.
- Score uniform and unigram baselines under identical evaluation conventions. The large-k limit should approach $\ln(4)$ for this toy model.
- Apply smoothing to every allowed cell, update the denominator, compare k on validation, and reserve test data for the final choice.
- Only then change the input to a larger names dataset. The invariants persist; its counts and losses will differ.

Understanding checks

1. Why does anna contribute five predictions? **Four characters plus END; START is the initial context.**
2. Does bigram mean two characters of memory? **No: one current token and one target form the pair.**
3. Why can sampling produce ana while greedy produces a? **Sampling can select n's 0.25 branch after a.**
4. Does one-in-sixteen guarantee one ana in sixteen attempts? **No: it is the expected frequency across repetitions.**
5. Why average NLL? **To report penalty per prediction instead of a total that accumulates with prediction count.**
6. Does smoothing fix an unknown character? **No: it redistributes probability only among defined outcomes.**
7. Can you generate before testing? **Yes. Testing measures a chosen version; it does not enable generation.**

References and provenance

The examples and explanations were developed for this series. The underlying n-gram estimation, likelihood, and smoothing concepts are established methods. Primary reading and educational context:

- Jurafsky, D. and Martin, J. H. Speech and Language Processing, Chapter 3, N-gram Language Models. <https://web.stanford.edu/~jurafsky/slp3/3.pdf>
- Bengio, Y., Ducharme, R., Vincent, P. and Jauvin, C. (2003). A Neural Probabilistic Language Model. JMLR 3, 1137-1155. <https://www.jmlr.org/papers/v3/bengio03a.html>
- Karpathy, A. makemore. Educational implementation and progression from character models to Transformers. <https://github.com/karpathy/makemore>